

Natural Computing

LMU Munich
summer term 2025

Thomas Gabor

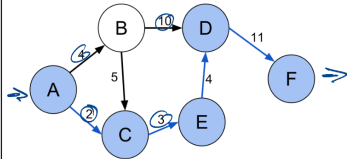


Definition 9 (optimization). Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{T}$ be an arbitrary set called target space and let $\leq$ be a total order on $\mathcal{T}$. A total function $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is called target function. Optimization (minimization/maximization) is the procedure of searching for an $x \in \mathcal{X}$ so that $\tau(x)$ is optimal (minimal/maximal). Unless stated otherwise, we assume minimization. An optimization run of length $g + 1$ is a sequence of states $\langle x_t \rangle_{0 \leq t \leq g}$ with $x_t \in \mathcal{X}$ for all t .

often: $\mathcal{T} = \mathbb{R}$

$\langle x_0, \dots, x_g \rangle$

Shortest Path



↳ example

$$x = \{ \underline{(A,C)}, \underline{(C,E)}, \underline{(E,D)}, \underline{(D,F)} \}$$

$$\mathcal{X} = \mathcal{P}(E) = 2^E$$

for graph edges E

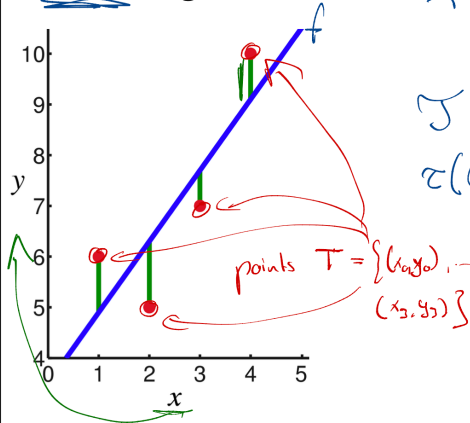
$$\mathcal{Y} = \mathbb{Z}$$

$$\tau = \{e_1, \dots, e_n\} \mapsto \sum_{i=1}^n \text{cost}(e_i)$$

$$\tau: \mathcal{X} \rightarrow \mathcal{Y}$$

$$x^* = \underset{x \in \mathcal{X}}{\text{argmin}} \tau(x)$$

Linear Regression



$$\mathcal{X} = \mathbb{R}^2$$

so that $f(x) = \underline{ax} + \underline{b}$ for $\underline{x} = (x, 1)$

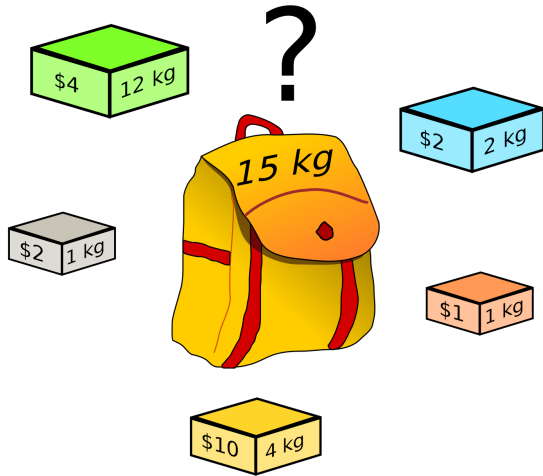
$$\mathcal{Y} = \mathbb{R}$$

$$\tau(a, b) = \sum_{i=0}^{n-1} |(ax_i + b) - y_i|$$

$$\mathcal{F} = \{f: \mathbb{R} \rightarrow \mathbb{R} \mid f \text{ is linear}\}$$

↳ mathematically possible
but very inefficient

The Knapsack Problem

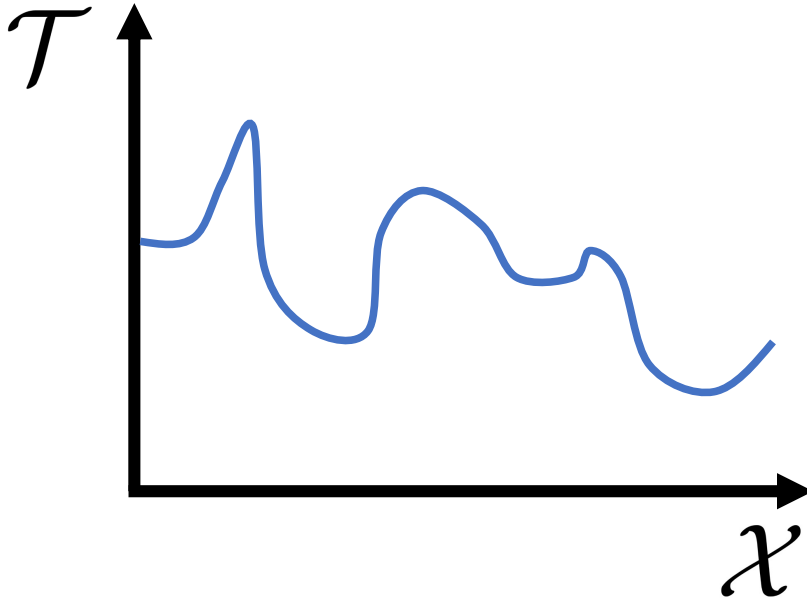


Playing Chess

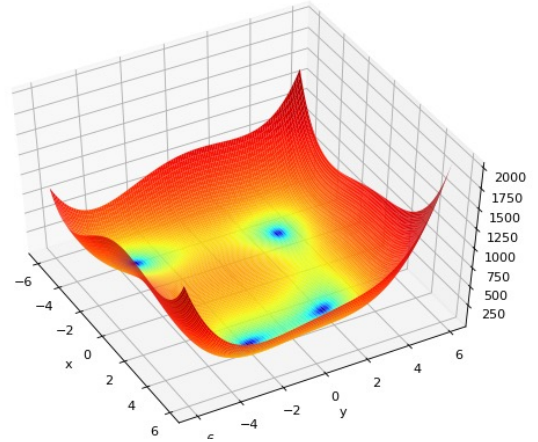
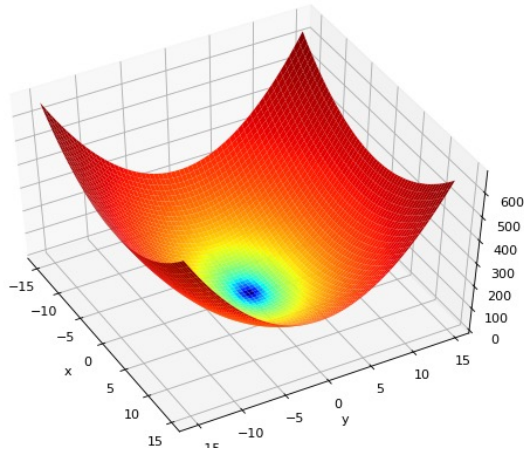


Further Examples

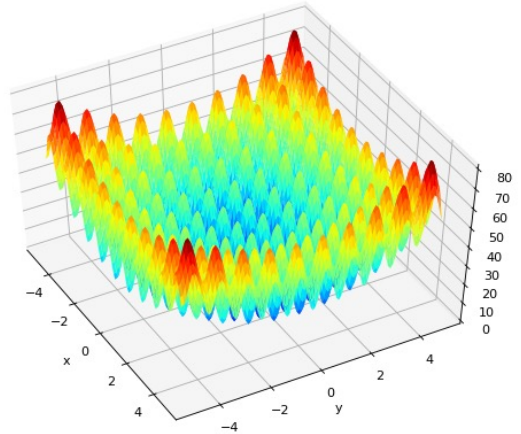
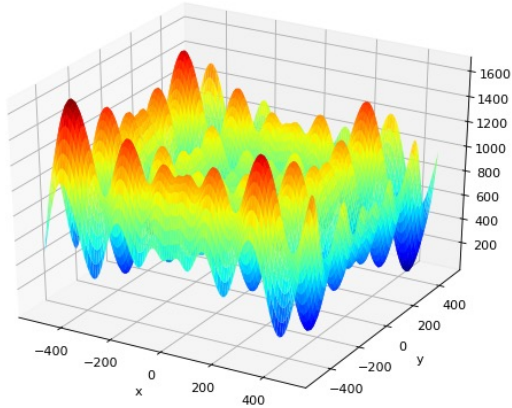
The Solution Landscape Metaphor



The Solution Landscape Metaphor



The Solution Landscape Metaphor



Definition 9 (optimization). Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{T}$ be an arbitrary set called target space and let $\leq$ be a total order on $\mathcal{T}$. A total function $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is called target function. Optimization (minimization/maximization) is the procedure of searching for an $x \in \mathcal{X}$ so that $\tau(x)$ is optimal (minimal/maximal). Unless stated otherwise, we assume minimization. An optimization run of length $g + 1$ is a sequence of states $\langle x_t \rangle_{0 \leq t \leq g}$ with $x_t \in \mathcal{X}$ for all t .

Definition 9 (optimization). Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{T}$ be an arbitrary set called target space and let $\leq$ be a total order on $\mathcal{T}$. A total function $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is called target function. Optimization (minimization/maximization) is the procedure of searching for an $x \in \mathcal{X}$ so that $\tau(x)$ is optimal (minimal/maximal). Unless stated otherwise, we assume minimization. An optimization run of length $g + 1$ is a sequence of states $\langle x_t \rangle_{0 \leq t \leq g}$ with $x_t \in \mathcal{X}$ for all t .

We use the following terminology:

- An optimization run is *working to some extent* if $t \leq t'$ for $t, t' \in [1; g] \subset \mathbb{N}$ correlates positively to some extent with $\tau(x_t) \geq \tau(x_{t'})$.
- An optimization run is *improving* if $\tau(x_t) \geq \tau(x_{t'})$ for all $t \leq t'$.
- An optimization run has *found a global optimum* iff $\tau(x) \geq \tau(x_g)$ for all $x \in \mathcal{X}$.

Let $e : \langle \mathcal{X} \rangle \times (\mathcal{X} \rightarrow \mathcal{T}) \rightarrow \mathcal{X}$ be a possibly randomized or non-deterministic function so that the optimization run $\langle x_t \rangle_{0 \leq t \leq g}$ is produced by calling e repeatedly, i.e., $x_{t+1} = e(\langle x_u \rangle_{0 \leq u \leq t}, \tau)$ for all t , $1 \leq t \leq g$, where x_0 is given externally (e.g., $x_0 =_{\text{def}} 42$) or chosen randomly (e.g., $x_0 \sim \mathcal{X}$). An optimization process is a tuple $(\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$.

We use the following terminology:

- The *end result* of an optimization process $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$ is x_g .
- The *best result* of an optimization process $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$ is

$$*\mathcal{D} = \arg \min_{\substack{x_t \\ 0 \leq t \leq g}} \tau(x_t)$$

with the *best target value* $|\mathcal{D}| = \tau(*\mathcal{D})$.

- An optimization process $\mathcal{D}' = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x'_t \rangle_{0 \leq t \leq g'})$ is a *continuation* of an optimization process $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$ iff $g' \geq g$ and $x'_t = x_t$ for all t with $0 \leq t \leq g$. We write $\ell(\mathcal{D})$ for the set of all possible continuations of $\mathcal{D}$, which can be large if e is randomized or non-deterministic.
- An optimization process $\mathcal{D}$ is *elitist* iff for all its continuations $\mathcal{D}' = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x'_t \rangle_{0 \leq t \leq g'}) \in \ell(\mathcal{D})$ it holds that $\tau(x'_t) \geq \tau(x'_{t'})$ for all $t \leq t'$.

How do we optimize for τ ?

Algorithm 1 (single-sample random search). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. The process $\mathcal{D}$ continues via (single-sample) random search if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \arg \min_{x \in \{x_t, x'_t\}} \tau(x)$$

where $x'_t \sim \mathcal{X}$ is drawn at random for each call to e .

Algorithm 2 (stochastic hill climbing). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. Let $neighbors : \mathcal{X} \rightarrow \wp(\mathcal{X})$ be a function that returns a set of neighbors of a given state $x \in \mathcal{X}$. The process $\mathcal{D}$ continues via stochastic hill climbing if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \arg \min_{x \in \{x_t, x'_t\}} \tau(x)$$

where $x'_t \sim neighbors(x_t)$ is drawn at random for each call to e .

Algorithm 3 (simulated annealing). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. Let $neighbors : \mathcal{X} \rightarrow \wp(\mathcal{X})$ be a function that returns a set of neighbors of a given state $x \in \mathcal{X}$. Let $\kappa : \mathbb{N} \rightarrow \mathbb{R}$ be a temperature schedule, i.e., a function that returns a temperature value for each time step. Let $A : \mathcal{T} \times \mathcal{T} \times \mathbb{R} \rightarrow \mathbb{P}$ with $\mathbb{P} = [0; 1] \subset \mathbb{R}$ be a function that returns an acceptance probability given two target values and a temperature. Commonly, we use

$$A(Q, Q', K) = e^{\frac{-(Q' - Q)}{K}}$$

for $\mathcal{T} \subseteq \mathbb{R}$ and Euler's number e . The process $\mathcal{D}$ continues via simulated annealing if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \begin{cases} x'_t & \text{if } \tau(x'_t) \leq \tau(x_t) \text{ or } r \leq A(\tau(x_t), \tau(x'_t), \kappa(t)), \\ x_t & \text{otherwise,} \end{cases}$$

where $x'_t \sim neighbors(x_t)$ and $r \sim \mathbb{P}$ are drawn at random for each call to e .

Algorithm 4 (gradient descent). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. Let $\mathcal{T}$ be continuous ($\mathcal{T} = \mathbb{R}$, e.g.) and let $\tau' : \mathcal{X} \rightarrow \mathcal{T}$ be the first derivative of τ . The process $\mathcal{D}$ continues via gradient descent (with update rate $\alpha \in \mathbb{R}^+$) if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = x_t - \alpha \cdot \tau'(x_t).$$

The learning rate α can also be given as a function, usually $\alpha : \mathbb{N} \rightarrow \mathbb{R}$ so that $e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = x_t - \alpha(t) \cdot \tau'(x_t)$. If τ is stochastic, this process is called stochastic gradient descent (SGD).

Algorithm 5 (simulated annealing with local gradient descent). For $n \in \mathbb{N}$, $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e_{\mathcal{D}}, \langle x_u \rangle_{0 \leq u \leq t})$ is an optimization process that continues via simulated annealing with n steps of local gradient descent if $e_{\mathcal{D}}$ has the form

$$e_{\mathcal{D}}(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \begin{cases} {}^*\mathcal{E}(x'_t) & \text{if } \tau({}^*\mathcal{E}(x'_t)) \leq \tau(x_t) \\ & \text{or } r \leq A(\tau(x_t), \tau({}^*\mathcal{E}(x'_t)), \kappa(t)), \\ x_t & \text{otherwise,} \end{cases}$$

where x'_t, r, A, κ are given as in Alg. 3 and where $\mathcal{E}(x'_t) = (\mathcal{X}, \mathcal{T}, \tau, e_{\mathcal{E}}, \langle x'_{t+v} \rangle_{0 \leq v \leq n})$ is an optimization process that continues via gradient descent (cf. Alg. 4) so that

$${}^*\mathcal{E}(x'_t) = \arg \min_{\substack{x'_{t+v}, \\ 0 \leq v \leq n}} \tau(x'_{t+v})$$

is the best solution found by $\mathcal{E}$ when starting at x'_t .

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t .

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$v_0 \leftarrow 0$ (Initialize 2nd moment vector)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)

end while

return θ_t (Resulting parameters)

How do these algorithms work?

Local vs. Global Information

Smoothness

Deception

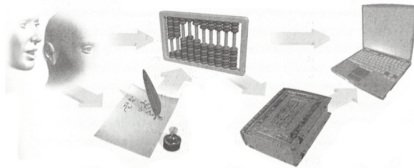
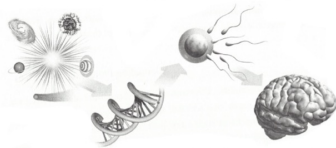
The Exploration/Exploitation Dilemma

Which algorithm do we choose?

No Free Lunch Theorem

Theorem 3 (no free lunch [1, 3]). As measured by sample efficiency, i.e., the achieved minimal value of τ per evaluations of $\tau(x)$ for some new $x \in \mathcal{X}$ for finite $\mathcal{X}$, all optimization algorithms perform the same when averaged over all possible target functions τ . So, for any search/optimization algorithm, any elevated performance over one class of problems is exactly paid for in performance over another class.

Artificial Chemistries



“The history of the universe can be thought of as a sequence of information processing revolutions, each of which builds on the technology of the previous ones.”

Seth Lloyd. *Programming the Universe*. Vintage Books, 2006.

On Multi-Sets...

Let X be a set with members from a space $\mathcal{X}$.

We can identify a set X with its membership function $\mu_X : \mathcal{X} \rightarrow \mathbb{B}$ such that

$$\mu_X(x) = 1 \iff x \in X.$$

For a multiset X' we generalize the membership function so that

$$\mu_{X'}(x) \geq 1 \iff x \in X'.$$

In both cases, the subset relation between (multi-)sets X, Y can be defined as

$$X \subseteq Y \iff \forall x \in X : \mu_X(x) \leq \mu_Y(x).$$

We then define $\wp(X) = 2^X = \mathbb{B}^X \cong X \rightarrow \mathbb{B}$,

and $\wp^*(X) = \mathbb{N}^X \cong X \rightarrow \mathbb{N}$,

to be the powerset functions for sets and multisets, respectively.

More on Multi-Sets...

Definition 10 (artificial chemistry, soup). Let $\mathcal{X}$ be a state space. Let $\mathcal{R}$ be a set of reaction rules $R \in \mathcal{R}$ where each R is a function $R : \mathcal{X}_R^{\text{dom}} \rightarrow \mathcal{X}_R^{\text{cod}}$ for $\mathcal{X}_R^{\text{dom}}, \mathcal{X}_R^{\text{cod}} \subseteq \wp^*(\mathcal{X})$. Let $\mathcal{A}(X, \mathcal{R})$ be the set of all *reaction materials* from $X \in \wp^*(\mathcal{X})$ for all rules in $\mathcal{R}$, i.e.,

$$\mathcal{A}(X, \mathcal{R}) = \{(X', R) \mid X' \subseteq X \wedge X' \in \mathcal{X}_R^{\text{dom}} \wedge R \in \mathcal{R}\}.$$

Let $A : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X}) \times \mathcal{R}$ be a possibly randomized or non-deterministic function that takes a multi-set $X \in \wp^*(\mathcal{X})$ and returns a multi-set of reactants and a reaction rule to perform so that $A(X) \in \mathcal{A}(X, \mathcal{R})$. The tuple $(\mathcal{X}, \mathcal{R}, A)$ defines an artificial chemistry. A multi-set $X \in \wp^*(\mathcal{X})$ of an artificial chemistry is also called soup; each element $x \in X$ is also called a particle. The evolution of a soup for g generations is a sequence $\langle X_0, \dots, X_g \rangle$ so that X_0 is given as the initial soup and, for all t with $0 \leq t \leq g - 1$

$$X_{t+1} = (X_t \setminus X') \uplus R(X')$$

where $(X', R) = A(X_t)$.

Definition 10 (artificial chemistry, soup). Let $\mathcal{X}$ be a state space. Let $\mathcal{R}$ be a set of reaction rules $R \in \mathcal{R}$ where each R is a function $R : \mathcal{X}_R^{\text{dom}} \rightarrow \mathcal{X}_R^{\text{cod}}$ for $\mathcal{X}_R^{\text{dom}}, \mathcal{X}_R^{\text{cod}} \subseteq \wp^*(\mathcal{X})$. Let $\mathcal{A}(X, \mathcal{R})$ be the set of all *reaction materials* from $X \in \wp^*(\mathcal{X})$ for all rules in $\mathcal{R}$, i.e.,

$$\mathcal{A}(X, \mathcal{R}) = \{(X', R) \mid X' \subseteq X \wedge X' \in \mathcal{X}_R^{\text{dom}} \wedge R \in \mathcal{R}\}.$$

Let $A : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X}) \times \mathcal{R}$ be a possibly randomized or non-deterministic function that takes a multi-set $X \in \wp^*(\mathcal{X})$ and returns a multi-set of reactants and a reaction rule to perform so that $A(X) \in \mathcal{A}(X, \mathcal{R})$. The tuple $(\mathcal{X}, \mathcal{R}, A)$ defines an artificial chemistry. A multi-set $X \in \wp^*(\mathcal{X})$ of an artificial chemistry is also called soup; each element $x \in X$ is also called a particle. The evolution of a soup for g generations is a sequence $\langle X_0, \dots, X_g \rangle$ so that X_0 is given as the initial soup and, for all t with $0 \leq t \leq g - 1$

$$X_{t+1} = (X_t \setminus X') \uplus R(X')$$

where $(X', R) = A(X_t)$.

Reaction rules are usually written in the form $R = \sum_i x_i \rightarrow \sum_j y_j$ for $x_i, y_j \in \mathcal{X}$, $i, j \in \mathbb{N}$, $\{x_i\}_i \in \mathcal{X}_R^{\text{dom}}$, $\{y_j\}_j \in \mathcal{X}_R^{\text{cod}}$, and $R(\{x_i\}_i) = \{y_j\}_j$.